

Atividade Prática: Sistema de Streaming "JavaPlay"

Cenário

Você foi contratado para desenvolver o núcleo de um sistema de streaming. O sistema lida com diferentes tipos de mídia (Filmes e Músicas) e precisa gerenciar como esses itens são reproduzidos, baixados e precificados.

Requisitos Técnicos

1. Abstração e Atributos Comuns

Crie uma **classe abstrata** chamada `Midia`.

- **Atributos:** `titulo` (String) e `duracaoEmMinutos` (int).
- **Construtor:** Deve inicializar os dois atributos.
- **Método Concreto:** `exibirDetalhes()`, que imprime o título e a duração.
- **Método Abstrato:** `double calcularCusto()`. Cada tipo de mídia tem uma regra de precificação diferente.

2. Contratos de Comportamento (Interfaces)

Crie duas interfaces para separar responsabilidades:

- **Reproduzivel:** Deve conter o método `void darPlay()`.
- **Baixavel:** Deve conter o método `void realizarDownload()`.

3. Especialização e Herança (Subclasses)

Implemente as seguintes classes:

- **Filme:**
 - Estende `Midia` e implementa `Reproduzivel` e `Baixavel`.
 - Atributo extra: `qualidade` (ex: "4K", "Full HD").
 - Regra de Custo: O custo base é R\$ 10,00. Se for "4K", soma-se R\$ 5,00.
- **Musica:**
 - Estende `Midia` e implementa apenas `Reproduzivel`.
 - Atributo extra: `artista` (String).
 - Regra de Custo: O custo é fixo em R\$ 2,00 por música.

4. Utilitários e Lógica Global (Método Estático)

Crie uma classe utilitária chamada `ConversorTempo`.

- Deve conter um **método estático** `formatarMinutos(int minutos)`.
- Esse método deve receber os minutos totais e retornar uma String formatada (ex: 125 min vira "2h 05min").

5. Execução e Polimorfismo

Crie uma classe `Main` que:

1. Instancie um `Filme` e uma `Musica`.
2. Utilize o método estático para exibir a duração formatada de ambos.
3. Crie um método chamado `processarPlayer(Reproduzivel item)` que chama o `darPlay()`. Use este método para testar o polimorfismo passando tanto o filme quanto a música.
4. Tente chamar o método de download para a música e observe o erro de compilação (já que música não implementa `Baixavel`).

Objetivos de Aprendizagem

Ao concluir esta atividade, o aluno deverá ser capaz de:

- Diferenciar quando um comportamento deve ser um método abstrato (precificação) ou uma interface (capacidade de download).
- Utilizar `super()` para reaproveitar construtores da classe pai.
- Entender que interfaces permitem tratar objetos diferentes (`Filme` e `Música`) de forma igual em contextos específicos (como no `Player`).
- Compreender que métodos estáticos servem para funções de suporte que não dependem do estado do objeto.